

Assume Breach

A First Course in Defensive Security

Michael Borck

2026-07-11

Assume Breach

A First Course in Defensive Security

Copyright © 2026 Michael Borck.

Published by Michael Borck
Perth, Western Australia

ISBN: (to be assigned)

First edition, in development.

This work is licensed under a Creative Commons Attribution (CC BY) licence. You are free to share and adapt the material for any purpose, provided you give appropriate credit.

AI disclosure: AI tools were used as thinking partners throughout the drafting, iterating, and refining of this book. The author reviewed, challenged, and took responsibility for every sentence.

Companion website: <https://michael-borck.github.io/assume-breach>

Source: <https://github.com/michael-borck/assume-breach>

Table of contents

Introduction: Assume Breach	1
Acknowledgments	7
I Part I · Foundations	9
1 Introduction to Information Security	11
2 Software Security & Malware	17
3 Authentication & Access	23
4 Cryptography	29
II Part II · Managing Risk	35
5 Risk Management	37
6 Incident & Disaster Planning	43
III Part III · Defending Systems & Networks	49
7 Network Security	51
8 VPN & Firewalls	57
9 Web Security & Data Protection	61
IV Part IV · People & the Evolving Threat	67
10 Cybercrime, Botnets & Forensics	69
11 Human Factors & Emerging Trends	75
V Part V · Putting It Together	79
12 The Long Game	81
Appendices	85
Appendix: Setting Up a Practice Environment	85

About the Author 87

Introduction: Assume Breach

The old question was “how do we keep attackers out?” The better one is “what do we do given that, sooner or later, they get in?” That shift in assumption changes everything.

The unlocked door behind the badge

Every organisation says its systems are secure. It puts a padlock icon on the login page, a privacy policy in the footer, and an “enterprise-grade encryption” line on the marketing site. Then, somewhere behind all of that, there is a server no one has patched in eleven months, a password four people share, and a staff member about to click a link they shouldn’t. The claim and the reality are two different things. The gap between them is where information security lives.

Modern security starts from an uncomfortable premise: **assume breach**. Assume the perimeter will fail, the patch will be late, and someone will click the link. Across a large enough system and enough time, they will. A defender who assumes they can keep every attacker out builds one wall and trusts it. A defender who assumes breach builds in depth, watches for the intrusion, limits how far it spreads, and plans to recover. This is not pessimism. It is how real systems are defended.

This book is a first course in that mindset. It will not turn you into a hacker or a penetration tester in twelve weeks. It will do something more useful: teach you to *think like a defender*. That means looking at a system and seeing, before anything goes wrong, where it could go wrong, what it would cost, and what to do about it. That habit of mind is the thing you take into any job, in any industry, for the rest of your career.

How to think like a defender

Defenders reason in a particular way, and the whole book is built to grow that habit. Assume nothing is safe until it is shown to be: a control that looks fine on paper is only a claim until you have reason to trust it. Understand the attacker in order to protect against them. You cannot defend what you do not understand, so we spend time on how attacks work, studying them to see what they need so you can take it away. Weigh the cost, because perfect security is unaffordable and unusable. Every control is a trade-off, and a defender's skill is deciding which ones are worth it. And never forget the human. Most breaches do not defeat the technology; they walk past it through a person. The human factor is often the whole story.

Who this is for, and how to read it

This is a first book on defensive security, for anyone meeting the subject for the first time: a student, someone moving into security from another part of IT, or a curious professional who wants the mental model without the jargon. It assumes no prior security background.

The chapters build in a deliberate order. The early ones establish the foundations: how security is measured, where software goes wrong, how identity and secrets are protected. The middle turns to managing risk and surviving incidents. The later ones defend systems and networks, and end where security always ends: with people. You will get the most from reading in order, because each chapter leans on the ones before it, but each is self-contained enough to revisit on its own.

The book carries the *concepts*. It stays tool-agnostic: it explains what is true on any system rather than which button to click, so it doesn't go out of date the moment a tool does. Where practice helps, a **Try it yourself** note points you at the kind of hands-on exercise the ideas invite, and the practice-environment appendix shows how to set up somewhere safe to do it. Each chapter ends with a few **Questions to consider**. They are prompts worth sitting with rather than a test. Several are easier to answer well *after* a later chapter, which is deliberate.

A word on using AI

Using AI to learn this material is a good idea, and this book encourages it. Ask an assistant to explain a concept a second way, to quiz you, or to challenge your answer to one of the end-of-chapter questions. What matters is that you stay in the conversation and do the thinking. Use it to understand faster. Do not use it to skip understanding. If you want to

get better at that skill itself, the companion volume *Conversation, Not Delegation* is about this: thinking *with* AI rather than delegating your thinking to it.

This book is open access under a Creative Commons Attribution (CC BY) licence. It is part of the books.borck.education series.

Copyright

Acknowledgments

This book began with the students I have taught. Information security is usually met first as a pile of definitions: CIA, controls, threats, frameworks. It takes a while before those pieces click into a way of thinking. The moment they do, when someone stops memorising the list and starts asking “so where would this actually fail?”, is the moment this book is written for. Their questions, and their confusion, shaped every chapter.

I am grateful to the colleagues who reviewed early drafts and pushed back where an explanation was still too clever or too vague. The book is clearer for it.

The open-source community behind Quarto, Pandoc, and GitHub made it possible to write, build, and publish this openly, with tools that are free and transparent.

How this book was written

This book was written with AI, using the method its companion volume describes. That companion, *Conversation, Not Delegation*, argues that AI is most useful as a thinking partner you converse with, not a tool you hand your thinking to. I wrote *Assume Breach* that way. AI helped draft, question, and refine. It also produced its share of confident, plausible mistakes, and catching those was part of the work. That is the point the book makes about security as well: the machine produces a draft, and the human supplies the judgement and takes responsibility. Every sentence here reflects the author’s judgement. The AI made the work faster. It did not do the thinking.

Part I

Part I · Foundations

Chapter 1

Introduction to Information Security

What are we protecting, from whom, and how would we even know if we'd succeeded?

This first chapter sets up the ideas the rest of the book builds on. There is nothing to install yet. Just read it, and hold on to the questions at the end. They are worth returning to as the later chapters give you more to answer them with.

1.1 What this chapter covers

By the end you should be able to:

- Define information security and explain the **CIA triad**: confidentiality, integrity, availability.
- Explain why those three pull against each other, and what “balancing CIA” means in practice.
- Say what a **security control** is, and what **defence in depth** buys you.
- Walk through the **phases of an attack**, and explain why a defender studies them.
- Distinguish **ethical** security work from malicious hacking, and see where the line sits.

1.2 Security is a property, not a product

You cannot buy security in a box. There is no single product, no switch, no certificate that makes a system “secure.” Security is not a thing a system *has*. It is a property of how the whole system behaves, including the people using it. A bank vault with the door propped open is not secure no matter how thick the steel. A perfectly configured server whose administrator uses “password1” is not secure either. Security is the emergent result of many choices, and it holds only as long as the weakest of them holds.

That is why this book keeps returning to a defender’s habit of mind rather than a checklist. Checklists go stale. The habit of asking “what am I protecting, from whom, and how would I know if it failed?” does not.

1.3 The CIA triad

Almost everything in information security can be traced back to protecting one of three properties of information. Together they are called the **CIA triad** (no relation to the agency).

Confidentiality is keeping information from those who should not see it. When you think of “security,” this is usually what comes to mind: secrets stay secret. Encryption, access controls, and passwords are all, at heart, confidentiality mechanisms. A confidentiality failure is a leak: data ending up in the wrong hands.

Integrity is keeping information correct and un-tampered. It is less obvious than confidentiality but often matters more. If an attacker cannot *read* your bank balance but can *change* it, you have a worse problem than a leak. Integrity mechanisms let you detect or prevent unauthorised change. A hospital needs to trust that a patient’s recorded blood type is the one a doctor entered, not one an attacker altered.

Availability is keeping information and systems there when they are needed. A system nobody can reach is no more useful than one that was deleted. Availability is the property attacked by a denial-of-service, threatened by a **single point of failure**, and protected by backups, redundancy, and capacity planning. It is the property people forget until the day the system is down and the business stops.

1.3.1 Balancing CIA

Here is the part that separates a defender from a checklist-follower: **the three properties work against each other**. You cannot turn all three up to maximum.

Maximise confidentiality (encrypt everything, demand three approvals

for every access) and you damage availability, because legitimate users now wait, forget keys, and get locked out. Maximise availability (copy the data everywhere, let anyone read it instantly) and you have made confidentiality much harder. Lock a system down so hard that no change is ever allowed, and you have protected integrity at the cost of a system too rigid to use.

Real security is about the right balance for this information, this organisation, this risk. The balance a hospital strikes for patient records is not the one a newspaper strikes for tomorrow's front page, and both are defensible. Learning to reason about that trade-off, rather than reaching for maximum everything, is the first real skill of the subject.

A fourth idea often sits alongside the triad: **non-repudiation**, the property that someone cannot credibly deny having done something. When a system can prove *who* made a change and *when*, disputes end and accountability begins. It is what turns "someone transferred the money" into "this account, at this time, authorised the transfer."

1.4 Controls and defence in depth

A **security control** is anything you put between an asset and a threat to reduce risk: a password, a firewall, a locked door, a policy, a training session, a backup. Controls come in familiar flavours: some **prevent** (a lock stops entry), some **detect** (an alarm notices it), and some **correct** (a backup restores what was lost). A good defence uses all three, because prevention eventually fails.

This is the idea the book is named for. **Defence in depth** means never trusting a single control to be enough. You assume each layer *will* fail, and you put another behind it. A castle did not rely on the outer wall alone. It had a moat, then a wall, then a gate, then an inner keep, so that breaching one defence still left the attacker facing the next. A modern system is the same: a firewall *and* patched software *and* least-privilege accounts *and* monitoring *and* backups. No one layer is trusted, so no one failure is fatal.

This is the practical face of **assume breach**. If you assume the attacker will get past your first control, you stop asking "is this wall strong enough?" and start asking "when this fails, what catches it?" That question builds better systems than the search for one perfect wall ever could.

1.5 The phases of an attack

To defend against attackers, you have to understand how they work. You are not learning this to imitate them. You study each step to see

what it *needs*, so you can take that away. Most intrusions move through recognisable phases:

1. **Reconnaissance.** The attacker gathers information about the target from the outside: who works there, what systems they run, what's exposed. Quiet, often legal, and easy to overlook.
2. **Scanning.** They probe the target directly for live systems, open ports, and running services, mapping the attack surface. (This is what you'll do, from the defender's side, in an early lab.)
3. **Gaining access.** They exploit a weakness (a flaw, a weak password, a tricked user) to get a foothold.
4. **Maintaining access.** They entrench, so a reboot or a changed password doesn't lock them out.
5. **Clearing tracks.** They cover their trail, deleting logs and hiding activity to delay discovery.

Read that list as a *defender* and each phase becomes an opportunity. Reconnaissance can be made harder by exposing less. Scanning can be detected. Access can be denied with patching and strong authentication. Persistence and track-clearing can be spotted by monitoring that the attacker cannot fully erase. Understanding the attacker's process is where a defender's leverage comes from.

1.6 Ethical hacking, and the line

Much of what you'll learn in this book is, in mechanism, identical to what an attacker does: scanning systems, cracking passwords, capturing traffic. The difference is **permission and purpose**. An ethical practitioner acts with the owner's authorisation, within an agreed scope, to *find and fix* weaknesses before a real attacker exploits them. The same action, without permission, is an offence.

That line is sharp and it matters. Practise only in an environment you own or are explicitly authorised to use. The skills are dual-use. Your professionalism is what makes them defensive.

Try it yourself

If you want to practise as you read, set up a safe, isolated environment now: a disposable virtual machine or a set of containers, kept off your main network, so later chapters' exercises have somewhere to run. Never practise on systems you do not own or have explicit permission to test. The practice-environment appendix lists some options.

1.7 Where this connects

The **CIA triad** and **defence in depth** are the lenses for the whole book. Almost every later control exists to protect one leg of the triad or to add one more layer. The next chapter begins the technical detail, looking at where software goes wrong and the malware that walks through the gaps.

1.8 Questions to consider

1. Give a real situation where **confidentiality** matters most, one where **integrity** matters most, and one where **availability** matters most. Use three different examples.
2. “More security is always better.” Describe one case where tightening security actually made things *worse*. What was traded away?
3. Why would a *defender* bother learning the phases of an attack? What specifically do you gain by understanding how an attacker thinks?

Chapter 2

Software Security & Malware

Most attacks don't break the maths. They exploit a mistake someone made writing the software, or a person willing to run something they shouldn't.

There is a comforting myth that attackers are code-breaking geniuses defeating unbreakable encryption. Reality is more mundane and more useful to understand. The overwhelming majority of attacks walk in through a flaw someone left in a program, or a user who ran a file they were tricked into trusting. This chapter is about those two doors: the bugs in software, and the malicious software that exploits them (or exploits us). Understand where the doors are, and you can start closing them.

2.1 What this chapter covers

By the end you should be able to:

- Distinguish **design flaws** from **implementation flaws**, and explain **attack surface**.
- Explain what **CVE** and **CWE** are, and how **vulnerability disclosure** works (responsible disclosure, zero-day).
- Classify the main families of **malware**: viruses, worms, spyware, adware, trojans, RATs, ransomware, rootkits, and botnets.
- Explain how a **trojan** and a **botnet** actually work, and how we detect and prevent them.
- Trace how a real breach chains several of these together.

2.2 Where flaws come from

Software fails in two ways, and the distinction matters because they are fixed differently.

A **design flaw** is a mistake in the plan. The software does what it was designed to do, and the design itself is insecure. A system that emails you your own password in plain text has no bug. It works as intended, and the intention is the problem. Design flaws are expensive because fixing them means changing the plan, often after everything has been built on top of it.

An **implementation flaw** is a mistake in the building. The design was sound, but somewhere in the code a programmer made an error: mishandled a length, trusted an input they shouldn't have, forgot a check. Most of the vulnerabilities you'll hear about are implementation flaws, and they are the reason software needs constant patching.

Every flaw is only a threat if an attacker can reach it. This brings us to **attack surface**: the sum of all the points where an attacker could try to get in. Every open port, every input field, every feature, every piece of installed software adds to it. A defender's instinct, seen already in the last chapter, is to *reduce* attack surface: turn off what you don't use, close what you don't need, and you shrink the number of doors an attacker can try. Attack surface is not fixed. It grows every time you add a feature, install software, or connect a new system, which is why security is never "done."

2.3 Naming the weaknesses: CVE and CWE

When there are millions of flaws across the world's software, you need a way to talk about them precisely. Two catalogues do this. A **CVE** (Common Vulnerabilities and Exposures) is a unique identifier for one *specific* vulnerability in one specific product: a particular flaw in a particular version, with a reference number everyone can point to. A **CWE** (Common Weakness Enumeration) is one level up: a category of *type* of flaw ("improper input validation," "use after free") that recurs across many products. CVEs are the individual cases; CWEs are the patterns behind them.

2.3.1 Disclosure and the zero-day

When a researcher finds a flaw, what should they do with it? The professional answer is **responsible (or coordinated) disclosure**: tell the vendor privately, give them reasonable time to build and ship a fix, and only then make the details public. This protects users during the

window when a fix doesn't yet exist. The alternative hands attackers a weapon: publishing a working exploit before a patch is available.

A **zero-day** is a vulnerability that is being exploited before the vendor knows about it (or before a patch exists). The defenders have had “zero days” to prepare. Zero-days are dangerous because the usual defence, patching, isn't available yet. They are also a reminder of why *assume breach* matters: some flaws will be exploited before anyone can fix them, so a defence that relies only on “keep everything patched” will, on some day, fail.

2.4 Malware: a field guide

Malware (malicious software) is the general term for any program written to do harm. The families are worth knowing because they spread and survive differently, and each difference is a place to defend.

2.4.1 Viruses and worms

A **virus** attaches itself to a legitimate file or program and spreads when that host is run and shared. It needs a carrier and, usually, a human to move it. A **macro virus** hides in the scripting built into documents (a spreadsheet or word-processor file), which is why “enable macros?” is a question worth treating with suspicion.

A **worm** is the more dangerous cousin: it spreads *on its own*, across the network, without needing a host file or a human to carry it. The 1988 Morris Worm, one of the first, showed the pattern that still holds: self-propagating code can move faster than any human response. For a defender, a worm's fuel is connectivity and unpatched systems, so segmentation and patching are what starve it.

2.4.2 Spyware, adware, and the willing click

Not all malware exploits a software flaw. **Spyware** collects information (keystrokes, browsing, credentials) and reports it back. **Adware** forces unwanted advertising and often tracks you to do it. A great deal of malware needs no vulnerability at all: it needs the user to *agree*, to click “install,” to run the attachment, to grant the permission. This is the theme that returns in the human-factors chapter: the most patched system in the world can still be undone by one willing click.

2.5 Trojans, RATs, and botnets

A **trojan horse** is malware disguised as something desirable: a game, a cracked application, a useful tool. The user runs it willingly, and while it may even do what it claims, it also does something else. A **Remote Access Trojan (RAT)** is a nasty variant: it hands the attacker remote control of the machine (files, webcam, keystrokes, the lot) as though they were sitting at it.

Scale this up and you get a **botnet**: a network of many compromised machines (“bots”) under one attacker’s control, taking orders from a command-and-control server. A botnet’s owner can rent it out to send spam, mine cryptocurrency, or overwhelm a target with a distributed denial-of-service (DDoS) attack. The infamous Zeus family, for example, turned huge numbers of ordinary PCs into a credential-stealing network. The defender’s insight is that a botnet’s value is *aggregate*. No single infected machine matters much, which is why individual users underestimate the cost of being compromised.

The modern headline act deserves its own mention: **ransomware** encrypts a victim’s files and demands payment for the key. It has become the dominant malware threat to organisations because it monetises a breach directly. It is where the recovery and continuity planning of a later chapter earns its keep. The defence that saves you is a tested backup, not a ransom payment.

2.6 Rootkits: hiding in the floor

A **rootkit** is malware that subverts a low level of the system (the operating system, or below it) so that it can hide everything above. By compromising the layer the rest of the system trusts to report the truth, a rootkit can make its own files, processes, and network connections invisible to the tools you’d normally use to find them. This is why compromised machines are often rebuilt from scratch rather than “cleaned”: once you can’t trust the floor you’re standing on, you can’t trust anything it tells you.

2.7 Defending against malware

No single control stops all of this. That is why the book argues for layered defence. Antivirus that matches known **signatures** catches yesterday’s malware but not a brand-new variant, so it must be one layer among many. Patching closes the flaws worms and exploits need. Least privilege stops a compromised program reaching everything. Controls limit what software is allowed to run. User education targets the willing click. Monitoring catches the intrusion that gets through anyway. “We have antivirus” is

not a security strategy. It is one thin layer, and a defender who relies on it alone has assumed the breach won't happen.

2.8 How it comes together: a real breach

Real attacks rarely use one flaw. They chain several. The 2013 Target breach is a classic worked example. Attackers first stole network credentials from a *third-party contractor* (a heating-and-cooling vendor with access to Target's systems). They used that foothold to reach the retail network and installed point-of-sale malware (the "BlackPOS" family) that scraped card data from checkout terminals. They exfiltrated tens of millions of card records while warning signs went unheeded. Read it as a defender and every stage is a control that could have caught it: tighter vendor access, network segmentation, restricted software execution, and someone acting on the alerts. No single fix would have stopped it. A chain of small failures let it through, and a chain of ordinary controls would have broken it.

Play it

Incident Zero, a free, print-and-play security game, has a **Hardening** module where you spend a limited budget on layered defences and then a pentester attacks. It's the fastest way to *feel* why one control is never enough, and why "we have antivirus" is not a security strategy. → incidentzero.retroverse.studio

2.9 Where this connects

The **introduction** laid out the phases of an attack, and this chapter fills in the "gaining access" phase. Stolen credentials, as in the Target breach, are the entry point the **authentication** chapter is about. Botnets and the criminal economy behind malware return in **cybercrime**, **botnets** and **forensics**.

2.10 Questions to consider

1. What's the difference between a **design flaw** and an **implementation flaw**? Give an everyday (non-software) analogy for each.
2. A worm and a trojan both spread, but they rely on different things. What does each one *need* to succeed, and which is easier to defend against?
3. Skim the Target breach. Write your one-sentence guess at the single change that would most likely have stopped it.

Chapter 3

Authentication & Access

A password isn't stored as a password. Understanding what's actually kept, and how it's attacked, is the whole of this chapter.

Almost every system begins with the same question: *who are you, and are you allowed to be here?* Getting that answer right is the foundation of confidentiality and integrity. If the system can't tell the real you from an impostor, none of its other protections mean much. This chapter is about how systems answer that question, the clever way they store the secret behind it, and the many ways attackers try to defeat both.

3.1 What this chapter covers

By the end you should be able to:

- Distinguish **identification**, **authentication**, and **authorisation**, and explain mutual authentication and ID proofing.
- Explain the **three factors** of authentication and what **multi-factor authentication** actually adds.
- Describe how a system **stores** credentials: cryptographic hashing, and why we add **salt**.
- Explain **Windows / NTLM** authentication at a high level and why it matters in practice.
- Analyse **password attacks** (dictionary, brute-force, rainbow tables) and the defences that beat each one: salting, logout, length.

3.2 Three words people mix up

These three terms are used loosely in everyday speech and precisely in security, and the precision matters.

Identification is a *claim*: you state who you are (a username, an account number, a name). It is just an assertion, and on its own it proves nothing.

Authentication is the *proof* of that claim: you demonstrate that you are who you said. Typing the password that matches the username is authentication. This is the step attackers most want to defeat, because a successful fake here makes everything downstream trust them.

Authorisation is what happens *after*: given that we now believe who you are, what are you allowed to do? A verified user is not an all-powerful one. Authentication gets you through the door. Authorisation decides which rooms you may enter. Keeping these separate lets a system trust *who* you are while still limiting *what* you can do. This is the principle of least privilege in action.

Two refinements are worth naming. **Mutual authentication** is when *both* sides prove their identity. Not only does the server check you, but you (or your software) check the server, so you're not handing your credentials to an impostor. **ID proofing** is the harder, real-world problem of establishing identity in the first place. How does a bank know the person opening an account is who they claim, *before* any password exists? Authentication verifies a claim you set up earlier. Proofing is how that claim was trusted to begin with.

3.3 The three factors

How can you prove who you are? Every method falls into one of three **factors**:

- **Something you know**: a password, a PIN, the answer to a question.
- **Something you have**: a phone, a hardware token, a smart card.
- **Something you are**: a fingerprint, a face, a voice (biometrics).

Passwords (“something you know”) dominate because they’re cheap and need no special hardware. They are also the weakest link in most systems. People choose guessable ones, reuse them everywhere, and hand them over when asked nicely. The usual responses help at the margins: complexity rules, minimum lengths, and **lockout policies** that stop an account after too many wrong guesses. But decades of evidence say a password alone is not enough. That is why the industry is moving toward **passwordless** methods, replacing the shared secret with something harder to steal or phish.

3.3.1 Multi-factor authentication

The strongest single improvement you can make is to require **more than one factor: multi-factor authentication (MFA)**. The power comes from *combining categories*: a password (know) plus a code from your phone (have) means stealing the password alone is no longer enough. The “have” factor takes many forms: a one-time code, a hardware **token** or key fob, or a **smart card** carrying a chip that proves possession. The “are” factor, biometrics, is convenient and hard to forget, but carries a permanent trade-off: you can change a breached password, but you cannot change your fingerprint. MFA is strong, but it is not magic. As the questions at the end ask you to consider, a determined attacker can still phish a one-time code or trick a user into approving a prompt. It raises the bar without removing it.

3.4 What a system actually stores

Here is the idea that surprises most people, and it is the hinge of the whole chapter: **a well-built system does not store your password**. If it did, anyone who stole the database (or any administrator) would know every user’s secret.

Instead, the system stores a **cryptographic hash** of your password. A hash function takes any input and produces a fixed-length fingerprint, with two useful properties: the same input always gives the same output, and you cannot run it backwards to recover the input from the output. So when you register, the system hashes your password and stores only the hash. When you log in later, it hashes what you typed and compares the two fingerprints. If they match, you knew the password, and the system verified you *without ever storing the password itself*. A stolen database of hashes is less catastrophic than a stolen database of passwords.

3.4.1 Windows and NTLM

You’ll see this in practice with Windows authentication. Rather than keep passwords, Windows stores password hashes and uses them in its logon process. The older **NTLM** (NT LAN Manager) scheme, still present for compatibility in many environments, is one such mechanism. The relevant point for a defender (and the reason password-cracking exercises use these) is that an attacker who reaches those stored hashes doesn’t have your password *yet*, but they have the raw material to try to recover it. That is what the next section is about.

3.5 Attacking passwords

If attackers can get the hashes but can't reverse them, how do they recover passwords? They don't reverse the hash. They **guess**, hash each guess, and look for a match.

A **dictionary attack** works through a list of likely passwords (common words, previously leaked passwords, predictable patterns) because real people rarely choose randomly. A **brute-force attack** tries every possible combination. It always works *eventually*, but “eventually” grows explosively with password length. That is why length beats complexity. The accelerator is a **rainbow table**: a giant precomputed lookup of hashes for enormous numbers of candidate passwords, so the attacker trades storage for speed and skips the recomputation.

3.5.1 The fix: salt

Rainbow tables have one fatal weakness, and the fix is elegant. Before hashing, the system adds a unique random value, a **salt**, to each password, and stores the salt alongside the hash. Now two users with the same password have *different* hashes, and the attacker's precomputed rainbow table, built without your specific salt, is worthless. Salting doesn't make any single password harder to *guess*. It makes the attacker's shortcut of precomputation impossible, forcing them back to slow, per-account work. It is a small change with an outsized effect, and its absence is one of the clearest signs a system was built without care.

3.6 The maths is rarely the weakest link

Everything above assumes the attacker has to *defeat* authentication. Often they go around it. A **keylogger** records the password as you type it, before any hashing happens. **Social engineering** just asks. A convincing email or phone call, and the user hands the secret over. **Sniffing** captures credentials sent in the clear across a network (which returns in the network-security chapter). And stealing the **password file** itself gives the attacker the hashes to work on at leisure. The recurring lesson of this book applies here: the cryptography is rarely what fails. The people and the plumbing around it are.

3.7 A word on single sign-on

One more idea you'll meet: **single sign-on (SSO)** lets you authenticate once and gain access to many systems, and **reduced sign-on** is a lighter version of the same convenience. Users love it, and it improves security in one way: fewer passwords means less reuse and fewer sticky notes. But it

concentrates risk: that one login now unlocks everything, so it *must* be protected accordingly. That is a large part of why SSO and strong MFA almost always travel together.

3.8 Where this connects

The hashing this chapter relies on is the subject of the next chapter, **cryptography**. Sniffing for credentials reappears in **network security**, when we capture packets on the wire. And stolen credentials, as in the Target breach, are how many intrusions begin, a thread running back to **software security and malware**.

3.9 Questions to consider

1. If a website is breached, why is it so much better for you that they stored a **salted hash** of your password rather than the password itself?
2. MFA asks for a second factor. Give a realistic example where MFA still fails, where the attacker gets in anyway. What does that tell you about relying on it?
3. Why does adding **salt** defeat a rainbow table, when it doesn't make any single password harder to guess?

Chapter 4

Cryptography

The security of a good cipher rests on the key, not on hiding how it works. That single idea, Kerckhoff's principle, organises the whole field.

Cryptography is the closest thing security has to a superpower: done right, it lets you protect information even when your enemy can see everything you send. It is also the part of the field most wrapped in mystique and most often misunderstood. This chapter strips away the mystique. You do not need the mathematics to reason about cryptography well. You need to understand what each kind of cryptography *guarantees*, what it *doesn't*, and where the real weaknesses lie (which, as ever, are rarely in the maths).

4.1 What this chapter covers

By the end you should be able to:

- Explain the difference between an **algorithm** and a **key**, and state **Kerckhoff's principle**.
- Distinguish **symmetric (one-key)** from **asymmetric (two-key)** cryptography and say what each is good for.
- Explain **hashing** vs **encryption**, and where digital signatures fit.
- Reason about attacks: brute-forcing small key spaces, known-plaintext and chosen-plaintext attacks.

4.2 The algorithm and the key

Every cipher has two parts, and telling them apart is the foundation of everything else. The **algorithm** is the method: the fixed procedure for scrambling and unscrambling. The **key** is the secret input that steers it: the particular setting that turns readable *plaintext* into scrambled *ciphertext* and back. The same algorithm with a different key produces different output. The set of all possible keys is the **key space**, and its size is one measure of a cipher's strength: the more keys an attacker would have to try, the better.

Here is where beginners' intuition usually goes wrong. It feels safer to keep the *algorithm* secret too. Surely hiding the method helps? **Kerckhoff's principle** says the opposite, and it is one of the most important ideas in the whole field:

A cryptosystem should be secure even if everything about it, except the key, is public knowledge.

In other words, the security must rest **entirely on the secrecy of the key**, never on the secrecy of the algorithm. This sounds counter-intuitive until you see why it's a *strength*. A secret algorithm has never been examined by anyone but its creators, so no one knows whether it's flawed. Once it leaks (it always leaks), your security evaporates. A public algorithm, published and attacked by the world's cryptographers for years, has earned its trust the hard way. This is why every cipher you should trust (the ones protecting your web traffic, your messages, your disk) has an openly published algorithm and keeps only the key secret. "Secret sauce" cryptography is a warning sign.

4.3 Symmetric cryptography: one shared key

The oldest and simplest arrangement is **symmetric** (or one-key) cryptography: the *same* key both encrypts and decrypts. If you and I share a secret key, I encrypt with it, you decrypt with it, and anyone without it sees only noise. Symmetric ciphers are fast and efficient, which is why they do the heavy lifting of bulk encryption: encrypting a whole disk, a database, or a stream of network traffic.

Symmetric cryptography has one stubborn problem, and it's not the maths: **key distribution**. Before we can communicate securely, we both need the same secret key. But how do I get it to you *securely* in the first place? If we could already send secrets safely, we wouldn't need the encryption. For two people who have never met, sharing a key over an untrusted network is the problem the whole system was meant to solve.

For centuries this was cryptography's central limitation.

4.4 Asymmetric cryptography: a key you can publish

The breakthrough that solved it is **asymmetric** (or two-key) cryptography. Instead of one shared secret, each person has a *pair* of mathematically linked keys: a **public key** they can hand out to anyone, and a **private key** they keep to themselves. The crucial property is that what one key locks, only the *other* can unlock. Knowing the public key does not let you work out the private key.

That single trick does two different jobs, and it's worth keeping them apart. The first is confidentiality. Anyone can encrypt a message to you using your *public* key, but only your *private* key can decrypt it. Now a stranger can send you a secret without ever sharing a key in advance. The key-distribution problem dissolves, because the encrypting key was never secret. The second is authenticity, through digital signatures. Run it the other way: you encrypt (sign) something with your *private* key, and anyone can verify it with your *public* key. Since only you have your private key, a valid signature proves the message came from you and wasn't altered. This is where non-repudiation and integrity come from.

Asymmetric cryptography is slower than symmetric, so real systems use **both**: asymmetric cryptography to safely exchange a fresh symmetric key, then fast symmetric encryption for the conversation. That hybrid is, in essence, what happens every time your browser opens a secure connection. The web calls this scheme Transport Layer Security, or TLS.

4.5 Hashing: one-way, and not encryption

A **hash function** is often grouped with cryptography but does something different, and the difference matters. Encryption is *reversible*. The whole point is that the right key gets the original back. A cryptographic hash is deliberately **one-way**: it turns any input into a fixed-length fingerprint that cannot be reversed to recover the input. You met this in the last chapter, where systems store the hash of a password rather than the password itself.

So the two answer different questions. **Encryption protects confidentiality**: keeping content secret from those without the key. **Hashing protects integrity**: proving that content hasn't changed, since any alteration produces a different fingerprint. If you want to hide a message, encrypt it. If you want to prove a file hasn't been tampered with, hash it.

Confusing the two is a common and revealing mistake: “I hashed it so it’s secret,” or “I encrypted it so I’ll know if it changed.”

4.6 Attacks, and the limits of weak crypto

Understanding how ciphers *fail* sharpens the intuition. The simplest failure is a **key space too small to matter**. The classic Caesar cipher shifts each letter by a fixed amount. But there are only twenty-five possible shifts, so an attacker tries them all in seconds. A **monoalphabetic substitution** cipher, where each letter maps to another, has an enormous key space yet still falls easily, because it leaks the *structure* of the language: letter frequencies survive the substitution, and ‘e’ is still the commonest letter. A big key space is necessary but not sufficient. A cipher can be broken by the patterns it fails to hide.

Cryptographers also reason about what an attacker gets to *see*. In a **known-plaintext attack**, the attacker has some plaintext *and* its matching ciphertext and uses the pair to attack the key. In the stronger **chosen-plaintext attack**, they can feed in plaintext of their choosing and study the output. A cipher worth trusting must withstand even an adversary with those advantages. That is why real ciphers are published and battered by experts rather than kept secret and hoped over.

Modern cryptography is strong, and attackers know it, so they almost never attack the maths. They steal the key from a poorly protected file, trick a user into revealing it, exploit a bug in the *implementation* of a sound algorithm, or take the data before it’s encrypted or after it’s decrypted. Even ransomware, which weaponises encryption *against* you, succeeds through the plumbing (how it gets in and what it can reach) rather than by breaking the cipher. The cryptography is the strongest link. Assume the attacker will go for the others.

4.7 Where this connects

The hashing that stores your password is the integrity side of **authentication**. The encrypted tunnel a VPN builds rests on the symmetric, asymmetric, and key-exchange ideas here, as **VPN and firewalls** will show. And the secure connection behind every reputable website is the hybrid scheme described above, which returns in **web security and data protection**.

4.8 Questions to consider

1. Why is it a *good* thing that strong ciphers publish their algorithms openly?
2. Symmetric crypto is faster; asymmetric solves key distribution. Sketch how a real system might use *both*.
3. What's the practical difference between **hashing** a message and **encrypting** it? When would you want each?

Part II

Part II · Managing Risk

Chapter 5

Risk Management

You can't protect everything equally, so you don't. Risk management is how a defender decides where the money and effort go.

Everything before this chapter was about *how* to defend: controls, cryptography, layered defence. This chapter is about *where to spend the effort*. It is the moment security stops being a technical subject and becomes a business one. No organisation has unlimited money, time, or patience, so a defender who tries to protect everything equally protects nothing well. Risk management is the discipline of deciding, honestly and defensibly, what to protect, how much to spend, and what to leave alone. It is also where “assume breach” turns into a budget.

5.1 What this chapter covers

By the end you should be able to:

- Define **risk**, **risk appetite**, and explain why appetite varies between organisations.
- Explain **cost-benefit analysis** and the difference between **intrinsic** and **acquired** value.
- Calculate **single loss expectancy (SLE)** and **annual loss expectancy (ALE)**, and use them to judge whether a control is worth it.
- Describe the main **risk treatments**: avoid, mitigate, transfer, accept.

5.2 What risk actually is

In everyday speech “risk” means danger. In security it means something more precise and useful: risk is the combination of **how likely** something bad is and **how much it would hurt** if it happened. A useful shorthand is $risk = likelihood \times impact$. Neither alone tells you much. A catastrophic event that almost never happens may be a smaller risk than a minor one that happens weekly. Treating them as equal, as a fear-driven approach does, wastes effort on the dramatic and ignores the routine.

Because risk is about likelihood and impact, it is always relative to a particular organisation. That is where **risk appetite** comes in: how much risk a given organisation is willing to accept in pursuit of its goals. A hospital and a social-media start-up face different threats, value different things, and answer to different regulators, so they will rationally accept different risks. Risk appetite is a deliberate statement of how much uncertainty an organisation will live with. It is neither recklessness nor timidity. Two well-run companies facing the same threat can make opposite, equally defensible decisions. That is the first question at the end of this chapter.

5.3 What are you protecting, and what’s it worth?

You cannot judge a risk without knowing the value of what’s at stake, and value is subtler than a price tag. An asset has **intrinsic value** (what it is worth in itself, like the replacement cost of a server) and **acquired value** (the worth it takes on from its role and context). A customer database might contain a few dollars of storage, but the acquired value (the trust, the regulatory exposure, the competitive harm if it leaked) can be enormous. Defenders who price only the intrinsic value under-protect the things that matter most, because the damage from losing information usually dwarfs the cost of the hardware holding it.

Once you can value what’s at stake, you can do **cost-benefit analysis**: weigh what a control *costs* against the loss it *prevents*. This is the engine of every good security decision. It has an uncomfortable implication people resist: sometimes the right answer is to *not* buy the control, because it costs more than the harm it prevents. Security is about protection that’s worth what it costs.

5.4 Putting numbers on it: SLE, ARO, ALE

To make cost-benefit concrete, security borrows a simple model from insurance. Three quantities do the work. **Single Loss Expectancy (SLE)** is how much one occurrence of the event would cost you. **Annualised Rate of Occurrence (ARO)** is how many times a year you expect it to happen, and it can be a fraction: 0.2 means once every five years. **Annual Loss Expectancy (ALE)** is the expected cost *per year*, $SLE \times ARO$.

Here is a worked example. Suppose a breach of your customer database would cost about **\$500,000** to clean up, notify, and recover from. That is the SLE. You judge such a breach might happen once every five years, so the ARO is **0.2**. The annual loss expectancy is:

$$ALE = SLE \times ARO = \$500,000 \times 0.2 = \$100,000 \text{ per year.}$$

That \$100,000 is what this risk “costs” you in expectation each year, and it’s the number a control has to beat.

5.5 Is the control worth it?

Now you can judge a control honestly. Say better monitoring and encryption cost **\$40,000 a year** and would cut the breach rate to once every twenty years (ARO 0.05). The new annual loss expectancy becomes $\$500,000 \times 0.05 = \$25,000$. So the control *reduces* your ALE from \$100,000 to \$25,000, a benefit of **\$75,000 a year**, for a cost of \$40,000. It saves you \$35,000 net, so it is worth buying.

The rule is that simple, and that powerful: **a control is worth it when the reduction in ALE exceeds the cost of the control**. Turn the numbers around and the answer flips. A \$50,000 control that only reduces an ALE from \$30,000 to \$5,000 *loses* you money, and buying it out of a vague sense that “more security is better” is the mistake this chapter exists to prevent. (That is the second question at the end. Work it through.)

Two honest cautions come with the arithmetic. First, notice that even after the good control, \$25,000 of annual risk remains: this is **residual risk**, the risk left over after treatment. You almost never reduce risk to zero, and pretending otherwise is how organisations get surprised. Second, these numbers are *estimates*: SLE and ARO are educated guesses, and some impacts (a ruined reputation, harm to people, loss of life) resist being priced at all. Treat quantitative risk analysis as a discipline for thinking clearly. The numbers structure the judgement; they do not replace it.

5.6 The four things you can do with a risk

Once a risk is identified and sized, you have four options. They are the vocabulary every security professional shares:

- **Avoid.** Don't do the risky thing at all. Stop offering the feature, don't collect the data, close the exposure. This is the only way to reduce a risk to zero, and often impractical because the risky activity is the business.
- **Mitigate.** Reduce the likelihood or the impact with controls. This is most of what the rest of this book is about: patch, encrypt, segment, train, monitor. You rarely eliminate the risk; you shrink it to an acceptable size.
- **Transfer.** Make it someone else's problem to bear, most commonly through **insurance**, or by outsourcing the risky function to a provider who takes on the liability. You still own the consequences, but you've shifted the financial hit.
- **Accept.** Decide the risk is small enough, or the fix too costly, to live with it. This is a legitimate, deliberate choice, *provided* it's made consciously and by someone with the authority to make it. Accepting a risk you've measured is professional; ignoring one you never looked at is negligence.

Real organisations do this at scale using structured frameworks (the **ASD Essential Eight**, **ISO/IEC 27001**, the **NIST Cybersecurity Framework**) that package hard-won judgement about which controls address which risks, so that every organisation doesn't have to reinvent the analysis. Underneath the framework, though, the logic is always the one in this chapter: size the risk, weigh the control, choose a treatment, and be able to defend the choice.

Play it

*Incident Zero's **Audit & Compliance** module has you score an organisation's controls pass/fail against a real framework, and then defend your judgement to the table. It turns the abstract choice between *accept*, *mitigate*, *transfer*, *avoid* into a call you have to argue for out loud. → incidentzero.retroverse.studio*

5.7 Where this connects

The next chapter, **incident and disaster planning**, is what you do when a risk you accepted or mitigated happens anyway. Every control elsewhere in the book is a *mitigation*, and this chapter is how you decide which ones are worth deploying. A maturity assessment against a standard like

the Essential Eight is this same risk-based reasoning, applied control by control.

5.8 Questions to consider

1. Two companies face the same threat but treat it differently. One buys insurance, one ignores it. How can both be *rational*?
2. A control costs \$50k/year and reduces an ALE of \$30k/year to \$5k/year. Should they buy it? Show your reasoning.
3. Give a real risk your own organisation (or somewhere you've studied or worked) probably **accepts** rather than fixes. Why is that a defensible choice?

Chapter 6

Incident & Disaster Planning

The question isn't whether something will go wrong, but whether you're ready when it does.

This is the chapter where *assume breach* stops being a slogan and becomes a plan. Everything so far has been about lowering the odds: hardening, controls, cryptography, spending where the risk is highest. But the honest premise of this whole book is that, sooner or later, across a large enough system and enough time, a defence fails and something gets through. The organisations that survive that moment are not the ones that believed it couldn't happen; they're the ones that decided *in advance* what they would do when it did. Planning for failure is the most practical form of optimism there is.

6.1 What this chapter covers

By the end you should be able to:

- Distinguish **incident response**, **business continuity**, and **disaster recovery**.
- Explain **RTO** and **RPO** and why they drive the plan.
- Describe what a good **incident response** process looks like (detect → respond → recover → learn).
- Explain the role of **backups**, alternate sites, and testing the plan.

6.2 When prevention runs out

A defender who assumes they can keep every attacker out will, reasonably, put every dollar into prevention and none into what happens after. That is a trap. Prevention has a ceiling. Zero-days exist, people slip, controls fail. Past that ceiling, the only thing that helps is *preparation to respond*. This is why the security lifecycle has a second half: because you assumed breach, you build detection, response, and recovery *before* you need them, when you're calm and thinking clearly, rather than inventing them at 3 a.m. during the worst day of the company's year.

6.3 Incident response: the lifecycle

An **incident** is a security event that matters: a confirmed compromise, a breach, an outage caused by an attack. **Incident response** is the disciplined process of handling it, and a good process runs through recognisable stages:

- **Detect.** You cannot respond to what you cannot see, which is why monitoring and logging, covered in the network chapter, exist. The faster you detect, the less damage is done. The time an attacker sits undiscovered (“dwell time”) is often measured in weeks, and every day of it costs.
- **Respond.** Contain the incident before it spreads: isolate the affected systems, cut the attacker's access, stop the bleeding. This is triage: the goal is to limit the damage, not yet to understand every detail.
- **Recover.** Restore normal operations safely: rebuild compromised systems (remember the rootkit: deeply compromised machines are rebuilt, not cleaned), restore data, and verify the attacker is gone before reconnecting.
- **Learn.** This is the stage everyone skips and no one should. Afterwards, honestly review what happened, what worked, and what didn't, then feed those lessons back into your defences. An incident you learn nothing from is one you've paid for twice.

Two things make or break response in practice, and neither is technical. **Roles** must be decided in advance (who declares an incident, who talks to whom, who has the authority to take a critical system offline), because a crisis is the worst possible time to be working out the org chart. And **communication** must be planned: internal coordination, and the harder question of what you tell customers, regulators, and the public, and when.

6.4 Business continuity and disaster recovery

Two related disciplines sit alongside incident response, and people constantly confuse them.

Business continuity is the big picture: keeping the *organisation* running during a serious disruption of any kind, not just a cyberattack. If the office floods, the payroll system dies, or ransomware locks every file, how does the business keep serving customers and paying staff? It covers people, processes, premises, and priorities, the whole organisation's ability to function through a crisis.

Disaster recovery is the technical subset of that: restoring the *IT systems and data* after they've been lost or damaged. Business continuity asks "how does the company keep operating?"; disaster recovery asks "how do we get the systems and data back?" Recovery serves continuity (you restore the systems so the business can continue), but they are not the same plan, and an organisation needs both.

6.5 The two numbers that drive the plan: RTO and RPO

Disaster recovery turns on two deceptively simple targets, and getting them straight is the most useful thing in this chapter.

Recovery Time Objective (RTO) is *how long you can afford to be down*: the maximum tolerable time to get a system back before the harm becomes unacceptable. A trading platform's RTO might be minutes; an internal wiki's might be days.

Recovery Point Objective (RPO) is *how much data you can afford to lose*: the maximum tolerable gap between your last good backup and the moment of failure. An RPO of one hour means you can lose at most an hour's work; an RPO of a day means losing a day's is acceptable.

The RPO, in particular, *dictates your backup schedule*. This is where plans contradict themselves. If a company declares an RPO of one hour but backs up only once a night, the plan is broken on its face: a failure just before the nightly backup could lose nearly twenty-four hours of data, not one. The stated goal and the actual practice don't match, and no one notices until the day it matters. (That mismatch is the second question at the end. It's a trap worth internalising, because real organisations fall into it constantly.)

6.6 Backups and alternate sites

Backups are the foundation of recovery, and the durable rule of thumb is **3-2-1**: keep at least **three** copies of important data, on **two** different types of media, with **one** kept off-site (or offline). The off-site, offline copy is the one that matters most against modern threats. Ransomware that can reach your backups can encrypt them too, and a backup the attacker can also destroy is no backup at all. This is the concrete payoff of the ransomware threat from the malware chapter: the organisation with good, tested, isolated backups can refuse to pay and restore; the one without them is at the attacker's mercy.

For systems that can't tolerate much downtime, recovery relies on an **alternate site**, a second location ready to take over. These come in flavours by cost and readiness: a **hot site** is fully equipped and running, ready in minutes; a **cold site** is just space and power you'd have to build out, cheap but slow; a **warm site** sits in between. Which you choose is a risk-and-cost decision: the cost-benefit reasoning of the previous chapter, applied to recovery.

6.7 The clock and the crowd

A modern breach is a legal and reputational problem as much as a technical one, and both run on clocks. Many jurisdictions now have **mandatory breach-notification** rules. The best-known, the EU's General Data Protection Regulation (GDPR), requires reporting certain breaches within **72 hours**. Recovery happens under a regulatory deadline, with penalties for silence. Meanwhile the *crowd* (customers, staff, media, regulators) wants to know what happened, and how you communicate under pressure can matter as much as the technical fix. Handled well, honest and timely disclosure preserves trust; handled badly, a cover-up turns a breach into a scandal.

The sharpest version of this pressure is the **ransom decision**: pay the attacker for the decryption key, or refuse? It looks like a simple cost calculation, but it is not. Paying funds criminal enterprises and may be illegal, offers no guarantee the data comes back, and marks you as willing to pay again. Refusing may mean a longer, costlier recovery, bearable only if your backups are good. There is rarely a clean answer, which is why it must be thought through *before* the moment arrives, not decided in a panic with the clock running.

6.8 Testing the plan

One rule outweighs all the rest: **an untested plan is only a hope**. Recovery plans fail at the moment they're needed for depressingly ordinary reasons: the backups were running but never restored-tested and turned out to be corrupt; the recovery documentation was itself on the system that went down; the one person who knew the process had left. Every one of these is caught by *rehearsing*: restoring from backups for real, running tabletop exercises, walking the team through the crisis while it's still hypothetical. The organisations that recover smoothly are the ones that practised; the ones that discover their plan's flaws during the real disaster paid the worst possible price for the lesson.

Play it

Incident Zero has two modules that live in this chapter. **Incident Response** races you to uncover a hidden attack chain before time and budget run out; **Disaster Recovery** drops you into a live breach with a 72-hour notification clock and the ransom decision on the table. Reading about incident response is one thing; running one against a clock is another. → incidentzero.retroverse.studio

6.9 Where this connects

This chapter is what you do when a risk from **risk management** happens anyway, and the hot, warm, and cold-site choice is that chapter's cost-benefit reasoning applied. Once an incident is contained, the investigation of *how* it happened is a discipline of its own, taken up in **cybercrime, botnets and forensics**. And the breach-notification obligations here are grounded in the data-protection law of **web security and data protection**.

6.10 Questions to consider

1. What's the difference between **business continuity** and **disaster recovery**? Give an example of each.
2. A company backs up nightly. Their RPO is "no more than one hour of data lost." What's wrong, and what would you change?
3. Why do so many disaster-recovery plans fail at the moment they're needed? What single practice would most reduce that?

Part III

Part III · Defending Systems & Networks

Chapter 7

Network Security

Everything you send crosses wires you don't control. This chapter is about what's visible there, and what to do about it.

Up to now we've mostly defended individual systems. But systems rarely stand alone. They talk to each other across networks, and every conversation crosses cables, switches, routers, and radio links you do not own and cannot see. That is a shift in the threat model. On a network, an attacker doesn't have to break into your machine to learn your secrets. They may only have to *listen*. This chapter is about how attackers map and eavesdrop on networks, why a surprising amount of traffic gives itself away, and what turns an open wire back into a private one.

7.1 What this chapter covers

By the end you should be able to:

- Explain how **scanning** and **sniffing** work and what they reveal.
- Describe what a **packet capture** shows, and why unencrypted traffic is so revealing.
- Describe common network attacks (**spoofing**, **man-in-the-middle**) and their defences.
- Explain why **encryption in transit** matters, and what segmentation and monitoring add.

7.2 The network as an attack surface

Recall the phases of an attack from the first chapter: reconnaissance, then scanning, then gaining access. Networks are where the first two of those

live. Before an attacker can exploit a system, they have to *find* it and learn what it's running. A network, by its nature, answers questions. Ask a machine “are you there?” and, unless told otherwise, it politely replies. Ask “what services are you running?” and its open ports announce them. Every device you connect adds to the attack surface, and every one of them is, by default, a little too willing to talk to strangers.

7.3 Scanning: mapping the target

Scanning is how an attacker (or a defender auditing their own network) turns a vague target into a map. It usually happens in layers. First, find the live hosts: a sweep that asks every address on a network “are you there?” and notes who answers. Then, for each live host, find the open **ports**. A port is a numbered doorway, and each open one usually corresponds to a running **service** (a web server, a file share, a remote-login service). Finally, probe those services to learn *which software and version* is behind them, because a known version can be matched against known vulnerabilities from the software-security chapter.

The result is a blueprint: here are your machines, here are the doors that are open, here's the software behind each one. To an attacker, that's a shopping list. To a defender, the same scan is an audit. That is why the tools are dual-use, and why the defensive instinct is to *close the doors you don't need*. Every service you're not using is a door you can shut, shrinking the blueprint an attacker gets to work from.

7.4 Sniffing: reading the wire

Scanning finds the doors. **Sniffing** reads what goes through them. A network interface normally ignores traffic not addressed to it, but put it into a listening (“promiscuous”) mode and it will capture *everything* it can hear: a **packet capture** of the raw conversations crossing the wire. A tool to read that capture then reassembles the packets into something human-readable.

One thing lands hardest the first time someone sees it: **a great deal of network traffic is sent in plain text, and sniffing shows all of it**. Older protocols were designed in a more trusting era and send everything unencrypted: the web's original HTTP, file transfer with FTP, remote login with Telnet, some email protocols. Capture that traffic and you don't just see *that* a conversation happened. You see its contents: the pages requested, the files moved, and the **usernames and passwords typed in the clear**. This is the sniffing that the authentication chapter warned about, made concrete: the strongest password in the world is worthless if it crosses the network in plain text for anyone listening to read.

It is unsettling to witness, and it converts “you should use encryption” from advice into an obvious necessity.

7.5 Attacks in the middle

Listening is passive. Network attacks get more dangerous when the attacker stops observing and starts *interfering*.

Spoofing is forging an identity on the network: sending traffic that claims to come from an address that isn’t yours. An attacker might spoof a trusted machine’s address to slip past a filter, or forge the source of an attack to hide their tracks. The network’s willingness to believe what packets *claim* about themselves is the weakness being exploited.

The more powerful move is the **man-in-the-middle** (or on-path) attack: the attacker positions themselves *between* two parties, so that everything each side sends passes through them first. Now they can read the conversation, and worse, *alter* it: changing a bank account number in transit, injecting malicious content, or capturing credentials as they flow past. Two common techniques get an attacker into that position on a local network. **ARP spoofing** tricks nearby machines into routing their traffic through the attacker by lying about which hardware address belongs to which machine. **DNS spoofing** answers a name lookup with the wrong address, so the victim connects to the attacker’s server while believing it’s the real one. The details matter less than the shape: the attacker exploits the small, trusting mechanisms that make a network convenient, and turns them into a wiretap.

7.6 Why the local attacker is special

Notice that ARP spoofing and casual sniffing both require the attacker to be *on the same network*. This is why an attacker who has gained a foothold on your local network, or who is sharing the coffee-shop Wi-Fi, can do things a purely remote attacker cannot. They can hear the local chatter, impersonate local machines, and slip into the middle of local conversations in ways that someone out on the wider internet, several routers away, cannot manage. It’s a concrete reason the *inside* of a network can never be treated as automatically safe. This is the assume-breach mindset applied to your own LAN. (That difference is the second question at the end; it’s worth being able to explain crisply.)

7.7 Defences: making the wire private again

The good news is that the central weakness, traffic readable by anyone listening, has a decisive fix, and you already met it.

Encryption in transit is the answer to sniffing. When traffic is encrypted (the web's HTTPS, which is HTTP wrapped in the TLS scheme from the cryptography chapter), an eavesdropper still captures the packets but sees only unreadable ciphertext. The credentials, the page contents, the files: all gibberish to anyone without the key. This single change neutralises passive sniffing. That is why the migration of the web from HTTP to HTTPS was one of the most important security improvements of the last decade, and why sending anything sensitive over an unencrypted protocol is now indefensible.

Two more layers back it up. **Segmentation** divides a network into zones with controlled routes between them, so that a foothold in one part doesn't grant free movement across the whole. It's the same idea that starves a worm (from the malware chapter) and contains an incident (from the planning chapter). **Monitoring** watches network traffic for the tell-tale signs of trouble (a host scanning its neighbours, traffic to a known-bad address, a volume spike), so that the intrusion you couldn't prevent is at least *detected*, feeding straight into the incident response of the previous chapter. Encryption hides the contents, segmentation limits the movement, and monitoring catches what gets through. No single one is enough.

Play it

Incident Zero's **Network Building** module makes you buy an architecture you can't fully afford, and the gaps you leave come back to bite in later rounds. It's a quick, concrete lesson in why security is a budget problem as much as a technical one. → incidentzero.retroverse.studio

7.8 Where this connects

The credentials a sniffer harvests are the ones **authentication** taught you to protect, and encryption in transit is what stops them leaking. That encryption is the hybrid TLS scheme from **cryptography**, in action. And the next chapter, **VPN and firewalls**, covers the two defences that build on this one: a firewall to control what traffic is allowed, and a VPN to protect it end to end.

7.9 Questions to consider

1. Name one everyday thing you do online that would be **visible in plaintext** to someone sniffing the network, and one that wouldn't. What's the difference between them?
2. Why can an attacker on the *same* network do things a purely remote attacker can't?
3. Imagine you captured your own device's traffic for one minute. What do you think you'd be able to read, and what would be hidden? What does that difference tell you about how you should send sensitive information?

Chapter 8

VPN & Firewalls

A firewall decides what traffic to let through. Writing a good rule is harder than it sounds, and that difficulty is the lesson.

The last chapter left us with two problems: networks let attackers *reach* systems they shouldn't, and they let attackers *read* traffic they shouldn't. This chapter is about the two classic defences that answer those problems. A **firewall** controls what traffic is allowed to pass. It decides who gets to reach what. A **VPN** protects traffic in transit. It makes the readable wire private again. Both are everyday tools, and both are widely misunderstood. Understanding what each does *and does not* do is worth more than knowing how to configure either.

8.1 What this chapter covers

By the end you should be able to:

- Explain what a **firewall** does and the difference between packet-filtering, stateful, and application-layer firewalls.
- Reason about **firewall rules**: default-deny vs default-allow, and why order matters.
- Explain what a **VPN** provides (confidentiality and integrity over an untrusted network) and what it does *not*.
- Describe **defence in depth** in a network context: no single control trusted alone.

8.2 What a firewall does

A **firewall** is a checkpoint. It sits at a boundary between networks (classically between a trusted internal network and the untrusted internet)

and inspects the traffic trying to cross, allowing what's permitted and blocking the rest. That's the whole idea: a gate with a guard who checks each arrival against a set of rules. A firewall does not try to block everything. Firewalls run at network boundaries, on individual hosts, and increasingly in cloud environments, but the job is always the same: decide, packet by packet or connection by connection, what's allowed through.

The difficulty, and the reason this chapter exists, is that the guard is only as good as the rules they're given. A firewall doesn't understand *intent*; it applies your policy literally. So the security a firewall provides is the quality of the thinking that went into its rules. It's a recurring truth in this book: the tool is easy, the judgement is hard.

8.3 The three kinds of firewall

Firewalls differ in how deeply they look at traffic, and the progression is worth knowing because each level trades speed for insight.

A **packet-filtering** firewall is the simplest: it examines each packet's headers in isolation (source and destination address, port, protocol) and permits or blocks based on those alone. It's fast, but it has no memory. Because it looks at each packet on its own, it can't tell whether a packet is a legitimate reply to a request you made or an unsolicited probe from an attacker; both look the same in isolation.

A **stateful** firewall fixes that by tracking the *state* of connections. It remembers that you opened a connection to a web server, so it knows to allow that server's replies back in, while still blocking unsolicited inbound traffic you never asked for. This connection awareness makes it more useful, and it's the modern default for most purposes.

An **application-layer** firewall (or proxy) goes deeper still: it understands the actual protocol (the structure of an HTTP request, for instance) and can make decisions based on the *content*, not just the addresses. A web application firewall that blocks a malicious-looking request is working at this level. It's the most powerful and the slowest, because understanding traffic costs more than routing it.

8.4 Writing the rules

The single most important decision in a firewall policy is its **default stance**, and it comes in two flavours. **Default-deny** blocks everything and permits only what you explicitly allow: a guest list. **Default-allow** permits everything and blocks only what you explicitly forbid: a banned list. Default-deny is more secure and is the professional's choice, for a subtle but decisive reason: with a banned list you can only block the

threats you *thought of*, and attackers make their living on the ones you didn't. A guest list fails safe: anything unforeseen is denied by default. A banned list fails open, letting the unforeseen straight through. The trade-off is friction: default-deny means every legitimate need must be anticipated and allowed, which is more work and more support tickets. That tension, security versus convenience, is the first question at the end, and it never goes away.

Two more subtleties bite people constantly. **Order matters:** firewalls evaluate rules top to bottom and usually act on the first match, so a broad “allow” placed above a specific “deny” silently defeats the deny. The packet matches the allow first and is waved through before the deny is ever considered. And **specificity matters:** one rule written too broadly can open a hole nobody notices, and a large rule set becomes so complex that mistakes hide in it. The cost of a bad rule is not abstract; a single misordered or over-broad line is how real networks end up exposed while everyone believes the firewall has them covered.

8.5 VPNs: a private tunnel over a public network

A firewall controls *what can pass*. A **VPN** (virtual private network) protects *what's already passing*. It solves the plaintext problem from the last chapter by building an encrypted **tunnel** between your device and a VPN server: everything you send is encrypted before it leaves your machine and only decrypted at the far end. To anyone listening in between (the attacker on the coffee-shop Wi-Fi, the operator of an untrusted network), your traffic is just ciphertext. The VPN gives you confidentiality and integrity across a network you don't trust, which is what an untrusted wire otherwise denies you.

Under the hood, a VPN is the cryptography chapter made practical: it uses asymmetric cryptography to establish trust and exchange keys, fast symmetric encryption for the bulk of the traffic, and integrity checks to ensure nothing was altered in transit. If that hybrid scheme sounds familiar, it should. It's the same pattern that secures the web.

8.6 What a VPN does not do

VPNs are also the most over-sold tool in consumer security, so it's worth being precise about their limits. The second question at the end turns on this.

A VPN protects the link between you and the VPN server. It does **not** make you anonymous: your traffic still emerges from the VPN server

toward its real destination, and that destination still knows what you did there. It does **not** secure your device: malware already on your machine is unaffected by an encrypted tunnel. And it does **not** protect the leg *beyond* the VPN server. If that onward traffic is unencrypted, it's exposed again once it leaves the tunnel (which is why a VPN is not a substitute for HTTPS; you want both). A VPN also doesn't remove trust; it *relocates* it. You've stopped trusting the local network and started trusting the VPN provider, who can now see your traffic. That can be a good trade on hostile Wi-Fi and a bad one with a shady provider, but "a VPN makes you safe" is a marketing line rather than a security fact.

8.7 Defence in depth, again

Neither tool is sufficient alone, and by now you can predict the conclusion. A firewall controls what traffic is allowed but can't judge the intent behind traffic it permits. Malware happily tunnels out over the same port 443 your firewall allows for ordinary web browsing. A VPN protects traffic in transit but does nothing about what happens at either end. So they join the layers from the last chapter: the firewall decides what's allowed, the VPN protects what's in motion, segmentation limits how far a foothold spreads, and monitoring catches the malicious traffic that slips through a permitted door. Each covers a gap the others leave. The answer to "what catches what the firewall misses?" is always another layer rather than a single better wall.

8.8 Where this connects

A firewall and a VPN are the two defences against the scanning, sniffing, and man-in-the-middle attacks of the previous chapter, **network security**. A VPN's tunnel is the hybrid encryption scheme from **cryptography**, deployed. And the monitoring that backs up a firewall feeds the detection stage of incident response in **incident and disaster planning**.

8.9 Questions to consider

1. Should a firewall **default to deny** or **default to allow**? Argue the trade-off, and say which you'd choose for a company network and why.
2. A VPN "makes you anonymous / makes you safe." What does it actually protect, and what does it *not*?
3. Give one example of a second layer catching something a firewall alone would miss.

Chapter 9

Web Security & Data Protection

The web is where most of us are attacked, because it's where most of us are. And behind every web app is data someone is legally obliged to protect.

Almost everything now runs through a web browser: banking, work, government, shopping, social life. That makes web applications the single largest attack surface most organisations have. It is also the place where ordinary people are most likely to meet an attack. This chapter has two halves that turn out to be one story. The first is *how web applications get attacked*, the handful of flaws behind a huge share of real breaches. The second is *what the law requires of the data behind them*. When a web flaw leaks personal information, it stops being only a technical problem and becomes a legal one. A web vulnerability is also a privacy incident.

9.1 What this chapter covers

By the end you should be able to:

- Explain the most common web vulnerabilities (injection, **XSS**, broken authentication, session issues) at a working level.
- Trace how a web attack unfolds, step by step.
- Explain core **data-protection** obligations (privacy principles and mandatory breach notification) with the major regimes as examples.
- Connect web weaknesses to the data-protection consequences when they're exploited.

9.2 Why the web is the front line

A web application is, almost by definition, exposed: it exists to accept requests from strangers over the internet. Every input field, URL, and uploaded file is a place where untrusted data enters your system, and the browser on the other end is a universal client you don't control. The core weakness behind most web attacks is a single recurring mistake: **trusting input that should never be trusted**. Data arriving from a user is treated as if it were safe, and the attacker's craft is to make that data do something it was never meant to.

9.3 Injection: when data becomes commands

The archetype is **injection**, and its most famous form is **SQL injection**. A web application builds a database query using something the user typed, a search term or a login name. If the application glues the user's input into the query text, an attacker can type input that becomes *part of the command itself*. Instead of a username they submit a fragment that changes what the query does: turning "find the user named X" into "find every user," or worse, "delete the table." The database, unable to tell the developer's intent from the attacker's addition, obeys.

Injection generalises beyond databases. Any time untrusted input is fed into something that interprets commands (a database, the operating system, a template engine), the same flaw can appear. And the defence generalises too: **keep data and code separate**. Rather than build a command by pasting text together, use mechanisms (parameterised queries, prepared statements) that hand the input to the system explicitly *as data*, so it can never be mistaken for instructions. Validate what comes in, and give the application only the database privileges it needs, so a successful injection does less damage.

9.4 Cross-site scripting: running attacker code in your browser

Cross-site scripting (XSS) is injection aimed at the *other* users of a site rather than its database. It is worth understanding because it explains how an attacker ends up running code in *your* browser without ever touching your machine.

A site lets users submit content (a comment, a profile, a message) and displays it back to other users. If the site fails to treat that content as untrusted, an attacker can submit a small **script** instead of ordinary text. When another user views the page, their browser receives that script *as*

part of the trusted site's page and runs it in the site's context, with the victim's session. Now the attacker's code, executing as if it were the site itself, can read the victim's session token, act as them, redirect them, or deface what they see. The victim did nothing wrong except visit a page they had every reason to trust. (Tracing that path in your own words is the first question at the end. It is the clearest way to show you understand XSS.) The defence mirrors injection: treat all user content as untrusted, and **encode it on the way out** so the browser renders it as text to display rather than code to run.

9.5 Authentication and session weaknesses

The web has a structural quirk that creates its own class of attacks. The protocol underneath is **stateless**: each request stands alone, with no memory of the last. So after you log in, the server issues a **session token** (usually stored in a cookie) that your browser sends with every subsequent request to say "it's still me." That token *is* your authenticated session. Anyone who steals it can impersonate you **without ever knowing your password**. This is **session hijacking**, and it's why the sniffing and XSS attacks in this book are so dangerous: they're often after the token rather than the password. A related trick, **session fixation**, has the attacker plant a session identifier they already know, then wait for the victim to log in under it. The defences are concrete: send tokens only over HTTPS so they can't be sniffed, flag cookies so scripts can't read them and they don't leak, issue a fresh token at login, and expire sessions so a stolen one doesn't last forever.

9.6 The other side: protecting the data

Behind every one of these attacks is *data*. Increasingly it is personal data, and the law now requires its protection. Modern **data-protection** law sets out principles for handling personal information that recur across jurisdictions: collect only what you need and only lawfully; use it only for the purpose you stated; keep it accurate; **secure it**; don't retain it longer than necessary; and let people see and correct what you hold about them. Australia codifies these as the **Australian Privacy Principles (APPs)** under its Privacy Act; the EU's **GDPR** is another version of the same pattern; other countries have their own. The names differ; the shape is consistent. The point for a defender is that *security is written into the law*. Failing to protect personal data breaches a legal obligation.

9.6.1 When it breaks: breach notification

The law also dictates what happens when protection fails. Most modern regimes include a **mandatory breach-notification** duty: if personal data is compromised in a way likely to cause serious harm, the organisation *must* tell the regulator and the affected people, often within a tight deadline. Australia's **Notifiable Data Breaches (NDB)** scheme requires it; the GDPR sets a **72-hour** clock, which is why the incident-planning chapter's crisis ran against that deadline. These rules exist for two reasons: so affected people can *protect themselves* (change passwords, watch their accounts, freeze their credit) and to create an *incentive* for organisations to secure data properly, since a breach can no longer be buried. The second question at the end asks about this. Notification turns a private failure into a public, accountable one by design.

9.7 A web flaw is a privacy incident

A single injection or XSS flaw that spills a customer database is, in the same instant, a technical failure *and* a legal event: it triggers the notification clock, exposes the organisation to regulatory penalty, and damages the trust that is the acquired value from the risk chapter. This is why web security and data protection belong in one chapter. The vulnerability and the consequence are two faces of one event. A defender who sees only the code half, or only the legal half, misses how the whole thing plays out.

9.8 Where this connects

This chapter connects to several others. **Authentication** is central because the session tokens attacked here are how the web remembers you are logged in, and stealing one sidesteps the password. **Network security** matters because sniffing and man-in-the-middle attacks are common ways session tokens get stolen, which is why HTTPS matters. **Incident & disaster planning** shares the breach-notification clock here, the same one that drives a real breach response.

9.9 Questions to consider

1. In plain words, how does a **cross-site scripting** attack end up running an attacker's code in *your* browser, on a site you trust?
2. A company is breached and customer data leaks. Under a mandatory breach-notification scheme, what are they obliged to do, and why does that obligation exist at all?

3. If an attacker steals your active **session token**, what can they do that they couldn't with just your username, and what, if anything, can they still *not* do?

Part IV

Part IV · People & the Evolving Threat

Chapter 10

Cybercrime, Botnets & Forensics

Modern attacks are a business. Understanding the economics (who profits, and how) tells you where the defences have to go.

It's tempting to picture the attacker as a lone figure in a hoodie, breaking in for the challenge of it. That picture is decades out of date. The early viruses were about mischief and notoriety, but the threat that matters today is organised, professional, and about money. Cybercrime has grown into an economy, with suppliers, specialists, marketplaces, and its own currencies. Once you see it as an economy, the defences make more sense. You stop asking “who would do this?” and start asking “who *profits* from this, and how do I make it unprofitable?” This chapter is about the business behind the attacks, and about how investigators follow the money and the evidence back to its source.

10.1 What this chapter covers

By the end you should be able to:

- Explain what a **botnet** is, and how one is built and monetised.
- Describe the **cybercrime economy**: the division of labour behind modern attacks.
- Explain, at a high level, how a real botnet works (credential-stealing and IoT examples).
- Connect forensic **traces** to the attribution of an attack, and explain chain of custody.

10.2 Cybercrime is a business

The single most useful shift in thinking is to treat serious cybercrime as *commerce*. Attackers have costs, revenues, competitors, and customers. They reinvest profits, offer support, build reputations, and specialise. Specialisation is more efficient, just as in any legitimate economy. The rise of cryptocurrency gave this economy a way to move value that is hard to trace. The result is a mature underground market where almost any capability can be bought.

Seeing crime as business reframes defence. You don't have to make an attack *impossible*. You only have to make it *unprofitable*. Raise the attacker's cost, lower their expected return, and much of the volume moves to an easier target. A great deal of security is about not being the cheapest victim on the market.

10.3 The division of labour

The defining feature of the modern cybercrime economy is that **no single criminal does the whole job**. The work is split among specialists, each selling to the next. **Malware developers** write and sell the tools, often as polished kits with updates and customer support. **Initial access brokers** specialise in breaking in, then *sell the foothold*, access to an already-compromised company, to whoever wants to use it. **Infrastructure providers** rent out “bulletproof” hosting and the command servers that run attacks. **Ransomware-as-a-service** operators lease their ransomware to affiliates, who run the attacks and hand back a cut. **Money mules and launderers** turn stolen funds into clean, spendable cash.

This specialisation has three consequences, and they explain why cybercrime is so stubborn. It **lowers the barrier to entry**: you no longer need to be a skilled hacker to launch an attack, you can rent the capability. It makes the ecosystem **resilient**: arrest one malware author and the access brokers, hosts, and launderers all carry on, and a competitor fills the gap. And it **muddies attribution**: with so many hands touching an attack, working out who is *responsible* is hard. That resilience-through-specialisation is the second question at the end, and it's the reason takedowns rarely end a threat for long.

10.4 Botnets: crime at scale

A **botnet** is the workhorse of this economy: a network of many compromised machines (the “bots”) all under the remote control of one operator through a **command-and-control (C2)** channel. The operator infects

machines by whatever means works (a trojan, a worm, an exploit), and each new victim joins the fleet, waiting for orders.

The power, and the answer to the first question at the end, is that a botnet's value is **aggregate**. A single compromised home PC is nearly worthless to a criminal. A *million* of them, acting together on command, are a weapon and a revenue stream: they can blast a target off the internet with a **distributed denial-of-service (DDoS)** attack (rented out by the hour), send spam and phishing at vast scale, mine cryptocurrency on someone else's electricity bill, harvest credentials and card numbers, or serve as a haystack of proxies to hide the operator's location. This is why individuals underestimate the cost of being infected. *Your* machine may hold nothing worth stealing, but your machine plus a million others is the product being sold.

10.5 How they work: two patterns

Two well-known cases show the pattern without needing any perishable statistics.

The **Zeus** family was a banking trojan built to steal credentials. It infected huge numbers of ordinary PCs, watched for banking logins, and siphoned them off. When its source code leaked, it spawned a generation of variants. Successful malware becomes a *platform* that others build on.

Mirai showed a newer front. Instead of PCs, it infected **Internet-of-Things (IoT)** devices (home routers, security cameras, DVRs) by trying a short list of *default passwords* the owners had never changed. From those humble devices it assembled a botnet powerful enough to launch some of the largest DDoS attacks ever seen. The lesson is that **cheap, insecure, unattended devices with default credentials are now part of your threat model**. The humblest gadget on a network can become a weapon.

10.6 Following the trail: forensics and attribution

When an attack succeeds, the work shifts to investigation: **digital forensics**, the discipline of reconstructing what happened from the traces left behind. And attackers leave more traces than they'd like. System and network **logs** record activity. File **timestamps** reveal a sequence of events. **Email headers** record the path a message travelled, betraying an origin the sender tried to hide. **Document metadata** stores the author, the software, and the times a file was created and edited. From fragments

like these an investigator builds a **timeline**: a defensible account of who did what, and when.

Two ideas make forensics a discipline rather than a rummage. The first is **chain of custody**: evidence must be collected and handled so that its integrity is beyond question: who touched it, when, and how, in an unbroken record. Mishandle it and you can *know* who did it and still lose the case, because you can no longer prove the evidence wasn't altered. (This is the hard lesson the game module makes you feel.) The second is that **attribution is difficult**. The same division of labour and the same botnets that power attacks also hide the people behind them: traffic routed through a botnet appears to come from innocent victims, infrastructure is rented and disposable, and false clues are sometimes planted deliberately. Careful defenders talk about attribution in terms of *confidence*, not certainty.

Still, attackers are human and make mistakes. The thread an investigator pulls first is usually the one the attacker *forgot to hide*: a log they didn't clear, a username or server reused across jobs, a document's metadata, an email header. Which trace that is, and why it's the first thread to pull, is the last question at the end.

Play it

Incident Zero's **Forensics** module has you chase attribution and rebuild a timeline while protecting the chain of custody. Mishandle the evidence trail and you lose the case, even when you know who did it. → incidentzero.retroverse.studio

10.7 Where this connects

This chapter connects to several others. **Software security & malware** supplies the weapons this economy trades in: the trojans, worms, and ransomware from that chapter. **Network security** matters because a botnet's command-and-control and DDoS traffic are what network monitoring exists to spot. Forensics, the investigation stage in this chapter, follows on from the containment and recovery in **incident and disaster planning**.

10.8 Questions to consider

1. Why is a **botnet** so much more valuable to a criminal than a single compromised machine?

2. Modern cybercrime has a “division of labour”: different people write malware, sell access, rent servers, and cash out. Why does that specialisation make attacks so hard to stop?
3. What kind of **trace** does a careless attacker most often forget to hide, and why is that the thread an investigator pulls first?

Chapter 11

Human Factors & Emerging Trends

Most breaches don't defeat the technology. They walk past it, through a person. This is where the whole book comes back to earth.

Every chapter until now has been, at heart, about machines: flaws, controls, cryptography, networks, code. But security fails, more often than any of those, through *people*. You can patch a server; you cannot patch a person, and attackers know it. The most sophisticated defences in this book can be undone by one employee clicking one link, one help-desk worker resetting one password for a convincing caller, one executive approving one urgent transfer. This final chapter is about that reality: the place security lives or dies. It is also about how the ground is shifting as AI arrives on both sides of the fight.

11.1 What this chapter covers

By the end you should be able to:

- Explain why the **human** is so often the weakest link, and why blaming users is the wrong response.
- Describe **social engineering** and **phishing**, and the defences that actually work.
- Reason about how non-technical people make security decisions with the information they're given.
- Discuss **emerging trends**, including AI's double role as both a tool for attackers and a tool for defenders.

11.2 The weakest link, and why blaming the user fails

It's true, and repeated everywhere, that people are the weakest link in security. What's said less often is that "the users are the weakest link" is a *trap*: a framing that excuses bad security. When a well-crafted phishing email fools an employee, the instinct is to blame the employee: they should have known better. But look closer. They were doing their job, quickly, inside a system that made the wrong action easy, the right action unclear, and the difference between the two nearly invisible. Blaming the person lets the *system* off the hook. A team that blames users stops fixing the thing they could fix.

The professional stance is the opposite. People will make mistakes. That is a constant, and good security is designed around it rather than trying to train it away. This is *assume breach* applied to human beings: assume that, sooner or later, someone will be fooled, and build so the organisation survives it. That single shift, from "make users perfect" to "make mistakes survivable", is the difference between security that works and security that just apports blame after the fact. (Whose fault a successful phish really is, is the first question at the end; it's worth sitting with.)

11.3 Social engineering and phishing

Social engineering is hacking the human instead of the machine: manipulating a person into doing what the attacker wants. It works because it targets the mental shortcuts we all rely on to get through the day. An attacker invokes **authority** ("this is IT, we need your password"), **urgency** ("the account will be closed in an hour"), **fear**, **trust**, or plain **helpfulness**. The victim, trying to be efficient and cooperative, complies. They are usually behaving *well* by the ordinary rules of work; the attacker has pointed those good instincts in the wrong direction.

Phishing is social engineering at scale: the deceptive email or message, cast wide, that lures people into revealing credentials or running malware. **Spear-phishing** is the targeted, personalised version aimed at a specific person, and it is more convincing. These attacks succeed because they exploit how humans are built to work. Their victims are not foolish.

The defences that help are instructive, because the obvious one is the weakest. **Awareness training** matters, but you cannot train people to be perfect forever, and a defence that requires every human to be vigilant every time will eventually fail. The stronger approach is to *reduce the reliance on human vigilance*: use technical controls that make a mistake less catastrophic. Multi-factor authentication means a phished password

alone isn't enough, which is the point of the authentication chapter. Email filtering stops most lures before they arrive. Firm *verification procedures* for high-risk actions mean no single email can move money or hand over data without an independent check. Best of all is to **make the safe path the easy path**, so that doing the secure thing is also the convenient thing. The goal is a system in which an ordinary human error does not become a disaster, rather than a workforce that never errs.

11.4 Communicating risk to people who aren't technical

A quieter human-factors failure is that we push security *decisions* onto people who have no way to make them well. The classic example is the browser **certificate warning**. When your browser says a site's certificate can't be trusted, it is telling you something important. It could not verify you're talking to who you think you are, using the identity-proving cryptography from the cryptography chapter. That warning might be the only thing standing between you and a man-in-the-middle attacker.

And yet most people click straight through it. Why? Because the message is written in technical language they don't understand, it appears at the moment they're trying to do something else, and they've been conditioned by past warnings that felt like false alarms to treat it as noise. This is a *design* failure, not the user's. A life-or-death security decision has been handed to someone without the information or context to make it, in words that mean nothing to them. Good security gives people clear information and, wherever possible, doesn't force impossible judgements on them. (Why people click through, and what the warning really means, is the second question at the end.)

11.5 Emerging trends: AI on both sides

The ground under all of this is shifting. The largest force reshaping it is artificial intelligence, a tool for *both* sides.

For **attackers**, AI removes the human bottlenecks that used to limit social engineering. The tell-tale signs of a phishing email (clumsy grammar, awkward phrasing, generic greetings) vanish when AI writes flawless, personalised messages at limitless scale. AI can clone a voice or fake a video convincingly enough to impersonate an executive authorising a transfer, automate the discovery of vulnerabilities, and lower the skill needed to run an attack to almost nothing. The human-factors attacks this chapter is about get cheaper, faster, and more convincing.

For **defenders**, the same technology is a force multiplier: AI can sift

oceans of logs for anomalies, triage the flood of alerts that overwhelms human analysts, flag phishing, analyse malware, and extend a small security team's reach. Which side does it favour? The honest answer is that it lowers costs for both. But by industrialising deception, it empowers the human-centred attacks that were already the most effective. That is why the defences in this chapter matter *more* now, not less. What doesn't change is the fundamental: AI makes the attacks in this book cheaper and more convincing, so designing for fallible humans becomes the central task rather than a peripheral one. (Which side AI favours, and why, is the last question.)

11.6 Where this connects

This chapter connects back across everything before it. Any technical control in the book can be undone by a person, so human factors runs through all of security rather than sitting beside it. Authentication is the clearest case: MFA is a technical control that makes a human mistake survivable. And that same thread, the human running through every stage, is part of how the next chapter ties the whole book into a single lifecycle.

11.7 Questions to consider

1. "Users are the weakest link." Argue *against* this framing: whose fault is a successful phish, really?
2. When your browser shows a certificate warning, what is it actually telling you, and why do most people click through it anyway?
3. Give one way AI helps **attackers** and one way it helps **defenders**. Which side do you think it favours, and why?

Part V

Part V · Putting It Together

Chapter 12

The Long Game

You do not secure a system the way you paint a fence: once, and it's done. Security has no finish line. That is not bad news; it's the whole idea.

You've reached the end of the concepts. The temptation now is to leave with a checklist: a tidy list of controls to apply and boxes to tick. Resist it, because every chapter of this book has argued against the checklist. Confidentiality, integrity, and availability pull against each other, so there is no setting that maximises all three. Controls fail, so no single one can be trusted. People slip, so no technology closes the last gap. Threats evolve, so yesterday's defence is not tomorrow's. A checklist assumes the work can be finished. The honest picture is different: security is a **cycle you keep turning**, not a state you reach.

12.1 Joining the dots

Everything in this book fits into one continuous loop. Walk it once, and watch the chapters fall into place:

- You **design** a system and decide what matters: which information, which assets, which trade-offs you're willing to make. (*The CIA triad; network architecture.*)
- You **harden** it, layer by layer, knowing each layer will eventually fail. (*Defence in depth; software security; authentication; cryptography; firewalls; web security.*)
- You **detect**, because you assumed breach: you watch for the intrusion you could not prevent, on the wire and in the logs. (*Network security.*)

- You **respond** when it comes, reading the attack and containing it before it spreads. (*Incident response.*)
- You **recover**, keeping the business alive and meeting your obligations under the clock. (*Disaster recovery.*)
- You **investigate** what happened, how far it went, and who was behind it, without destroying the evidence in the process. (*Cybercrime and forensics.*)
- You **audit and learn**, measuring the gaps honestly and feeding what you find back into the design. (*Risk management; the assessment mindset behind frameworks like the Essential Eight.*)
- And running through every one of those stages is the **human**: the analyst making the call, the user clicking the link, the board deciding what risk to accept. (*Human factors.*)

Then you begin again, because the system changed, the threats changed, and the last turn of the loop taught you something. That is the whole-of-life view: a set of things you keep *doing*, in order, forever, rather than a list of things to have.

12.2 Why “assume breach” is not defeatism

It might sound bleak to assume you’ll be breached. It isn’t. It’s the assumption that makes the loop turn. If you believed you could build one perfect wall, you would stop after hardening; detection, response, recovery, and investigation would all be wasted effort on a wall that never fails. It’s *because* you assume the wall fails that you build the rest. Assume breach is not surrender. It is the reason a defender’s job is a whole lifecycle and not a single act. And it is what separates security that survives contact with reality from security that only looks good on the marketing page.

Play the whole thing

Every stage in that loop is easier to *understand* than to *do*. Doing it, under time and budget pressure, is where the judgement lives. *Incident Zero*, the free print-and-play game referenced throughout these chapters, has a full campaign that runs this entire lifecycle end to end: design a network, harden it, detect and respond to a breach, recover from the disaster, and investigate what happened. Playing it is the closest thing to *living* a security cycle in an afternoon. If this book gave you the map, the campaign lets you walk the ground.
→ incidentzero.retroverse.studio

12.3 The skill that matters

If there is one thing to carry out of a first course in defensive security, it is not the contents of any single chapter. It is the shape of the loop, and the discipline to keep it turning honestly. Anyone can learn a control. The defender's real skill is holding the whole lifecycle in view: designing for failure, watching for the breach, responding without panic, recovering with obligations met, learning from every incident, and starting the next cycle wiser than the last.

That's the long game. It has no finish line, and that is by design.

12.4 Questions to consider

1. Pick a system you use every day (your phone, a bank app, your workplace network). Walk it once around the loop: where is it *designed*, *hardened*, *watched*, and *recovered*? Which stage looks weakest?
2. "Assume breach" changes what you build. Name one thing a team would do differently if they assumed they'd be breached, versus one that assumed they could keep everyone out.
3. Of the eight stages in the loop, which do organisations most often neglect, and why do you think that particular one gets skipped?

Appendix: Setting Up a Practice Environment

The chapters teach concepts and stay tool-agnostic. If you want to *practise*, and you should because security is a craft, you need somewhere safe to do it. This appendix is about that, and only that.

The one rule

Only ever practise on systems you own or have explicit, written permission to test. Scanning, cracking, capturing traffic, probing a web app: these are the same actions an attacker takes. Doing them against systems you don't control is, in most countries, a crime regardless of intent. The whole point of a practice environment is that everything in it is yours to break.

What a good practice environment looks like

Whatever you use, aim for three properties:

- **Isolated.** Kept off your main network, so a mistake can't reach anything real. A host-only or NAT network is enough for most work.
- **Disposable.** Quick to reset to a clean state when an exercise leaves a mess, so you can experiment without fear.
- **Reproducible.** Easy to rebuild the same way twice, so your results mean something and a broken setup is no disaster.

Two common ways to get there

Virtual machines. Free hypervisors (for example VirtualBox) let you run a self-contained guest operating system in a window. Snapshot it

once it's set up; roll back to the snapshot whenever you want a clean slate. Heavier to move around, but familiar and graphical.

Containers. Tools like Docker package an environment as a small, scriptable image that starts in seconds and is described by a file you can version and share. Lighter than a VM and easily reproducible, and increasingly how security tooling is packaged and taught. Some graphical tools are exposed through a web browser, so you don't lose the GUI. If you're choosing today, this is the direction most of the field is moving.

Tools you'll meet

You don't need to install these to read the book; they're named so the concepts have concrete anchors. Free, widely used examples by capability:

Capability (from the chapters)	Common free tools
Scan a host for open ports and services	Nmap
Crack password hashes; run a dictionary attack	John the Ripper, Hashcat
Capture and read network traffic	Wireshark
Encrypt / decrypt; work with keys	OpenSSL, GnuPG
Configure and test a firewall	iptables / nftables
Recover and analyse forensic evidence	The Sleuth Kit, Autopsy, Volatility
Explore web vulnerabilities safely	Deliberately vulnerable apps (e.g. DVWA, Juice Shop)

Tools change; the capabilities don't. If a tool here has been superseded by the time you read this, the concept it served is still in the chapter. Find the current tool for that capability and carry on.

About the Author



Figure 1: Photo of Michael Borck

Michael Borck is a software developer, educator, and auditor-by-training working at the intersection of human expertise and artificial intelligence. He teaches information security, and this book grew out of the introductory course he wanted to hand students. It builds the *defender's mindset* rather than a checklist of terms.

His book *Substantiate, Don't Assume* takes the same discipline into professional security auditing. *Conversation, Not Delegation* argues that AI is most valuable as a thinking partner you converse with rather than a tool you delegate to. *Assume Breach* sits at the entry point to that path: the first course, before the audit and before the profession.

Michael builds educational software and resources, and writes about the 80/20 principle in learning and professional judgement.

Connect

- michaelborck.dev: Professional work and projects
 - michaelborck.education: Educational software and resources
 - 8020workshop.com: Passion projects and workshops
 - [LinkedIn](#)
-

Other Books in This Series

Security Track:

- *Assume Breach: A First Course in Defensive Security* (this book)
- *Substantiate, Don't Assume: Why the Auditor's Opinion Matters More Than Ever in the Age of AI*

AI-Collaboration Track:

- *Conversation, Not Delegation: How to Think With AI, Not Just Use It*
- *Partner, Don't Police: AI in the Business Classroom*

Python Track:

- *Think Python, Direct AI: Computational Thinking for Beginners*
- *Code Python, Consult AI: Python Fundamentals for the AI Era*
- *Ship Python, Orchestrate AI: Professional Python in the AI Era*
- *Converse Python, Partner AI: The Intentional Prompting Methodology*

Web Track:

- *Build Web, Guide AI: Business Web Development with AI*